Django Signals Notes: Beginner to Advanced

Prepared by: Faruk Ahmed

Topics: Django Signals, Signal Receivers, Model Events, Transactions, Best Practices

Level: Beginner to Advanced

Language: English

Part 1: Introduction to Django Signals

1. What Are Django Signals?

Django Signals allow different parts of a Django application to communicate when specific events occur.

A signal can notify registered functions when an event happens, allowing those functions to perform additional tasks.

For example:

- When a new user is created, automatically create a user profile.
- When an order is created, send a notification.
- When a product is deleted, record an audit event.
- When an order is successfully paid, schedule a confirmation email.
- When a ManyToMany relationship changes, update related information.

Signals are based on an event-driven programming approach.

Simple Example

```
A new User is created
          |
          v
      post_save
          |
          v
   Signal Receiver
          |
          v
   Create User Profile
```

In this example, creating a user triggers the `post_save` signal. A registered receiver function then creates the associated profile.

2. Why Do We Use Django Signals?

Django signals are useful when an action should respond to an event without requiring the code that caused the event to directly call the receiver function.

Common use cases include:

1. Automatically creating related model instances.
2. Recording audit information.
3. Scheduling notifications.
4. Maintaining derived data.
5. Reacting to ManyToMany relationship changes.
6. Performing cleanup after model deletion.

Without Signals

```
user = User.objects.create_user(
    username="faruk",
    password="secure-password",
)

Profile.objects.create(user=user)
```

The code that creates the user must also explicitly create the profile.

With Signals

```
user = User.objects.create_user(
    username="faruk",
    password="secure-password",
)
```

A registered signal receiver can create the profile automatically.

Important: Signals are not always the best solution. For important business workflows, an explicit service function is often easier to understand, test, and maintain.

Part 2: Understanding Signal Components

1. Signal

A **Signal** is an event notification mechanism.

Examples of built-in Django signals include:

- `pre_save`
- `post_save`
- `pre_delete`
- `post_delete`
- `m2m_changed`
- `pre_init`
- `post_init`
- `request_started`
- `request_finished`

These signals notify registered receivers when their corresponding events occur.

2. Receiver

A **Receiver** is a function or callable that responds to a signal.

Example:

```
from django.db.models.signals import post_save
from django.dispatch import receiver



@receiver(post_save)
def my_receiver(sender, instance, created, **kwargs):
    print("A model was saved.")
```

This receiver listens to `post_save` signals from any sender because no sender restriction has been specified.

For a real application, you should usually connect a receiver to a specific model or signal to avoid executing it unnecessarily.

3. Sender

The **Sender** identifies the object or class responsible for sending the signal.

For model signals, the sender is generally the model class.

Example:

```
@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    print("A Product was saved.")
```

Here:

- `sender` is the `Product` model class.
- `instance` is the particular `Product` object being saved.
- `created` indicates whether the save created a new database record.

4. Instance

The `instance` argument represents the model object associated with the event.

For example:

```
@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    print(instance.name)
```

If a product named `Laptop` is saved, the receiver can access its name through `instance.name`.

5. The `created` Argument

The `created` argument is provided by `post_save`.

- `True`: A new database record was created.
- `False`: An existing database record was updated.

Example:

```
@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    if created:
        print("Product created.")
    else:
        print("Product updated.")
```

Remember: `created` is a `post_save` argument. It is not automatically available in every Django signal.

Part 3: Setting Up Django Signals

A common approach is to organize signal receivers in a separate `signals.py` file.

Suppose the application structure is:

```
myproject/
│
├── manage.py
│
```

```
├── myproject/
│   ├── settings.py
│   └── urls.py
│
└── accounts/
    ├── __init__.py
    ├── apps.py
    ├── models.py
    ├── signals.py
    └── views.py
```

1. Create a Model

In `accounts/models.py`:

```
from django.conf import settings
from django.db import models


class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="profile",
    )

    bio = models.TextField(blank=True)

    def __str__(self):
        return self.user.get_username()
```

This model creates a one-to-one relationship between a user and a profile.

2. Create `signals.py`

In `accounts/signals.py`:

```
from django.contrib.auth import get_user_model
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Profile
```

```
User = get_user_model()


@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

When a new user is created, the receiver creates a corresponding profile.

3. Register Signals in `apps.py`

In `accounts/apps.py`:

```
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"

    def ready(self):
        import accounts.signals
```

The `ready()` method is called when Django initializes the application registry. Importing `signals.py` registers the receiver.

Use the appropriate application import path for your project.

4. Register the Application

In `settings.py`:

```
INSTALLED_APPS = [
    # Other installed applications
    "accounts.apps.AccountsConfig",
]
```

Make sure the application configuration used by Django is the one that imports your signals.

Important: Register receivers during application initialization, not inside views or repeatedly executed functions. Repeated registration can cause duplicate signal execution.

Part 4: Important Django Model Signals

1. `pre_save`

The `pre_save` signal is sent at the beginning of a model's save operation, before the database save occurs.

Example

```
from django.db.models.signals import pre_save
from django.dispatch import receiver

from .models import Product



@receiver(pre_save, sender=Product)
def product_pre_save(sender, instance, **kwargs):
    print("Product is about to be saved.")
```

Common Use Cases

- Normalizing field values.
- Preparing data before saving.
- Inspecting the model's current in-memory state.

Important Limitation

`pre_save` does not mean that the database save will eventually succeed.

Avoid performing irreversible external actions, such as sending an email, before the save has succeeded.

---

2. `post_save`

The `post_save` signal is sent at the end of a model's save operation.

Example

```
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Product



@receiver(post_save, sender=Product)
```

```
    def product_post_save(sender, instance, created, **kwargs):
        if created:
            print("A new product was created.")
        else:
            print("An existing product was updated.")
```

Common Use Cases

- Creating related records.
- Recording model events.
- Scheduling follow-up operations.
- Triggering application notifications.

Important: `post_save` does not necessarily mean that the surrounding database transaction has been committed. If the transaction later rolls back, the database changes can be undone.

---

3. `pre_delete`

The `pre_delete` signal is sent at the beginning of a model deletion operation.

Example

```
from django.db.models.signals import pre_delete
from django.dispatch import receiver

from .models import Product


@receiver(pre_delete, sender=Product)
def product_pre_delete(sender, instance, **kwargs):
    print(f"Deleting product: {instance.name}")
```

Common Use Cases

- Recording deletion information.
- Capturing relevant data before deletion.
- Performing appropriate cleanup preparation.

The instance may still be available even though its database record is about to be deleted.

---

4. `post_delete`

The `post_delete` signal is sent at the end of a model deletion operation.

Example

```
from django.db.models.signals import post_delete
from django.dispatch import receiver

from .models import Product



@receiver(post_delete, sender=Product)
def product_post_delete(sender, instance, **kwargs):
    print(f"Product deleted: {instance.name}")
```

Common Use Cases

- Recording deletion events.
- Updating external systems.
- Cleaning up application-managed resources.

Important: Deleting a database record does not guarantee that an external side effect should happen immediately. If the surrounding transaction rolls back, the database deletion may be undone.

Also, deleting a model instance does not automatically mean that every associated file in media storage is deleted. File cleanup must be handled separately when required.

---

5. `pre_init` and `post_init`

These signals are sent before and after a model instance is initialized.

Example

```
from django.db.models.signals import post_init
from django.dispatch import receiver

from .models import Product



@receiver(post_init, sender=Product)
def product_initialized(sender, instance, **kwargs):
    print("Product instance initialized.")
```

These signals can be triggered when model instances are constructed, including when Django constructs instances from database query results.

Best practice: Avoid expensive database queries inside `pre_init` or `post_init`. They may execute very frequently.

Part 5: `pre_save` vs. `post_save`

Feature	`pre_save`	`post_save`
Execution time	Before the model save operation	At the end of the model save operation
Database save completed	Not yet	The save operation has completed
Receives `created`	No	Yes
Common use	Prepare or inspect data	Respond to a save event
Transaction committed	Not necessarily	Not necessarily

Simple Workflow

```
Model.save()
     |
     v
  pre_save
     |
     v
 Database Save
     |
     v
  post_save
```

This illustrates the ordinary model save lifecycle. The surrounding transaction may continue after `post_save`.

Part 6: Practical Example — Automatically Create a User Profile

This is one of the most common Django Signals interview examples.

1. Models

In `accounts/models.py`:

```
from django.conf import settings
from django.db import models


class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="profile",
    )

    bio = models.TextField(blank=True)
    phone = models.CharField(
        max_length=20,
        blank=True,
    )
```

```
        def __str__(self):
            return self.user.get_username()
```

2. Signal Receiver

In `accounts/signals.py`:

```
from django.contrib.auth import get_user_model
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Profile


User = get_user_model()


@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.get_or_create(
            user=instance
        )
```

3. Application Registration

In `accounts/apps.py`:

```
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"

    def ready(self):
        from . import signals
```

4. Usage

```
user = User.objects.create_user(
    username="faruk",
```

```
        password="secure-password",
    )
```

When the user is created, the signal receiver creates the associated profile.

The `get_or_create()` call provides additional protection against duplicate profile creation if a profile already exists.

However, it does not eliminate every possible concurrency issue or replace the database's one-to-one uniqueness constraint.

Best practice: If the profile is a required part of a business workflow, explicitly creating it in a service function may be more reliable and easier to test.

Part 7: Avoiding Recursive Signals

A common Django Signals mistake is causing a signal receiver to trigger itself repeatedly.

1. The Problem

Consider the following code:

```
@receiver(post_save, sender=Product)
def update_product(sender, instance, **kwargs):
    instance.name = instance.name.upper()
    instance.save()
```

What Happens?

1. A product is saved.
2. The `post_save` receiver executes.
3. The receiver calls `instance.save()`.
4. Saving the product triggers `post_save` again.
5. The receiver calls `instance.save()` again.
6. The cycle continues.

This can cause infinite recursion until the application fails.

2. A Safer Approach

If you need to update a field in the database, a direct queryset update can avoid triggering the model's `post_save` signal again.

```
@receiver(post_save, sender=Product)
def update_product(sender, instance, **kwargs):
    normalized_name = instance.name.upper()

    if instance.name != normalized_name:
        Product.objects.filter(
            pk=instance.pk
        ).update(
            name=normalized_name
        )
```

`QuerySet.update()` does not call the model's `save()` method and does not emit `pre_save` or `post_save` signals.

However, this operation also bypasses normal model save behavior. It should only be used when that is appropriate.

Another Option: Prepare the Value Before Saving

If the value can be normalized as part of the normal save operation, doing so before the database write may be simpler.

For example, a custom model method or explicit service function can normalize the value before calling `save()`.

Best practice: Do not call `instance.save()` inside a `post_save` receiver unless you have a carefully designed mechanism to prevent recursive execution.

Part 8: Django `m2m_changed` Signal

The `m2m_changed` signal is sent when a ManyToMany relationship changes.

For example, suppose a product can belong to multiple categories, or a student can enroll in multiple courses.

1. Example Models

```
from django.db import models


class Course(models.Model):
    title = models.CharField(max_length=100)
```

```
class Student(models.Model):
    name = models.CharField(max_length=100)

    courses = models.ManyToManyField(
        Course,
        related_name="students",
    )
```

2. Signal Receiver

```
from django.db.models.signals import m2m_changed
from django.dispatch import receiver

from .models import Student



@receiver(
    m2m_changed,
    sender=Student.courses.through,
)
def courses_changed(
    sender,
    instance,
    action,
    reverse,
    model,
    pk_set,
    using,
    **kwargs,
):
    if action == "post_add":
        print("Courses added.")

    elif action == "post_remove":
        print("Courses removed.")

    elif action == "post_clear":
        print("All courses removed.")
```

3. Using the Relationship

```
student = Student.objects.get(pk=1)
course = Course.objects.get(pk=1)

student.courses.add(course)
```

The signal receiver can respond to the relationship change.

4. Important `m2m_changed` Arguments

Argument	Meaning
`sender`	The intermediate relationship model
`instance`	The object whose relationship manager was used
`action`	The relationship operation being performed
`reverse`	Indicates whether the reverse

	relationship manager was used
`model`	The related model class
`pk_set`	The relevant set of primary keys for applicable actions
`using`	The database alias being used

5. Important Actions

Action	Meaning
`pre_add`	Before relationships are added

`post_add`	After relationships are added
`pre_remove`	Before relationships are removed
`post_remove`	After relationships are removed
`pre_clear`	Before relationships are cleared
`post_clear`	After relationships are cleared

Important: During clear operations, `pk_set` may be `None`. Do not assume it always contains the primary keys of affected objects.

Also, `m2m_changed` is associated with ManyToMany relationship-manager operations. Directly modifying the intermediate database table does not necessarily trigger the signal.

Part 9: Custom Django Signals

Django also allows you to create your own signals.

Custom signals are useful when you want to notify different parts of an application that a particular application-level event has occurred.

1. Define a Custom Signal

In `accounts/signals.py` or a separate `signals.py` module:

```
import django.dispatch


order_placed = django.dispatch.Signal()
```

2. Create a Receiver

```
from django.dispatch import receiver

from .signals import order_placed


@receiver(order_placed)
def handle_order_placed(sender, order, **kwargs):
    print(f"Order placed: {order.pk}")
```

3. Send the Signal

```
order_placed.send(
    sender=Order,
    order=order,
)
```

The registered receiver can now respond to the event.

Using `send_robust()`

Django also provides `send_robust()`:

```
responses = order_placed.send_robust(
    sender=Order,
    order=order,
)
```

Unlike `send()`, `send_robust()` catches exceptions raised by individual receivers and returns the exceptions as part of its response list.

However, neither `send()` nor `send_robust()` automatically creates a database transaction or makes external side effects reliable.

When Should You Use Custom Signals?

- When multiple independent components need to respond to an event.
- When you are developing a reusable Django application.
- When an application-level event needs to notify registered listeners.

Best practice: Avoid creating custom signals for every internal function call. A normal function call is often clearer when one component directly depends on another.

Part 10: `dispatch_uid` and Duplicate Registration

A receiver can accidentally be registered more than once in some application configurations.

The `dispatch_uid` parameter provides a unique identifier for a signal receiver registration.

Example

```
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Product



@receiver(
    post_save,
    sender=Product,
    dispatch_uid="product_post_save_handler",
)
def product_saved(sender, instance, **kwargs):
    print("Product saved.")
```

Django uses the `dispatch_uid` to identify the receiver registration for that signal and sender.

This can help prevent duplicate registrations when the same receiver is connected more than once with the same identifier.

Important: `dispatch_uid` does not prevent recursive signals or repeated saves. It only helps manage duplicate receiver registration.

Part 11: Transactions and `transaction.on_commit()`

One of the most important production-level signal concepts is the difference between a database save and a successful transaction commit.

1. The Problem

Consider this receiver:

```
@receiver(post_save, sender=Order)
def order_saved(sender, instance, created, **kwargs):
    if created:
        send_order_email(instance)
```

Suppose the order is created inside a transaction.

```
with transaction.atomic():
    order = Order.objects.create(
        # Order fields
    )

    raise ValueError("Something went wrong.")
```

The `post_save` receiver may send an email before the outer transaction is committed.

If the transaction later rolls back, the email may already have been sent even though the order no longer exists in the database.

2. Use `transaction.on_commit()`

Django provides `transaction.on_commit()` to register a callback that runs after the relevant transaction successfully commits.

```
from functools import partial

from django.db import transaction
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Order



@receiver(post_save, sender=Order)
def order_saved(sender, instance, created, **kwargs):
```

```
        if created:
            transaction.on_commit(
                partial(send_order_email, instance.pk)
            )
```

Here, `send_order_email()` is a placeholder function that should accept an order ID.

How It Works

```
Order Created
      |
      v
   post_save
      |
      v
Register on_commit Callback
      |
      v
Transaction Commits Successfully?
      |
      +------ Yes ------> Run Callback
      |
      +------ No -------> Discard Callback
```

If the transaction rolls back, the callback is not executed.

If there is no active transaction, the callback runs immediately.

Important Production Consideration

`transaction.on_commit()` prevents the callback from running before a successful commit, but it does not guarantee delivery of an email or execution of an external task.

For reliable delivery, consider a background task queue and, for critical workflows, a transactional outbox pattern.

Part 12: Signals and `QuerySet.update()`

A common source of confusion is that not every database modification triggers model signals.

1. Using `save()`

```
product = Product.objects.get(pk=1)
```

```
product.name = "Updated Laptop"
product.save()
```

This normally triggers `pre_save` and `post_save`.

2. Using `QuerySet.update()`

```
Product.objects.filter(
    pk=1
).update(
    name="Updated Laptop"
)
```

This updates the database directly.

It does not call each model instance's `save()` method and does not emit `pre_save` or `post_save`.

3. Using `bulk_create()`

```
Product.objects.bulk_create([
    Product(name="Laptop", price=50000),
    Product(name="Phone", price=20000),
])
```

`bulk_create()` does not call each object's normal `save()` method and does not emit the usual per-instance `pre_save` and `post_save` signals.

4. Using `bulk_update()`

```
products = list(Product.objects.filter(is_active=True))

for product in products:
    product.price += 100

Product.objects.bulk_update(
    products,
    ["price"],
)
```

`bulk_update()` does not call the normal `save()` method for each instance and does not emit the usual per-instance save signals.

Comparison Table

Operation	Normal model save signals
`instance.save()`	Yes
`Model.objects.create()`	Yes, for the instance save
`QuerySet.update()`	No
`bulk_create()`	No per-instance save signals
`bulk_update()`	No per-instance save signals
`instance.delete()`	Normally emits delete signals
`QuerySet.delete()`	Emits delete signals for

	deleted objects, subject to Django's deletion behavior

Remember: If your application's business logic depends on signals, using bulk operations or direct queryset updates may bypass that logic.

Part 13: Signals and File Handling

Signals are sometimes used to clean up files when a model is deleted.

For example, a product may have an image stored in media storage.

However, deleting a model does not necessarily delete its associated file.

Example

```
from django.db.models.signals import post_delete
from django.dispatch import receiver

from .models import ProductImage



@receiver(post_delete, sender=ProductImage)
def delete_product_image_file(sender, instance, **kwargs):
    if instance.image:
        instance.image.delete(save=False)
```

This requests deletion of the stored file when the `ProductImage` database record is deleted.

Important Considerations

- The file may be stored in external storage such as S3.
- File deletion may be irreversible even if the database transaction rolls back.
- Shared files may be referenced by other model instances.
- Storage deletion can fail independently of the database operation.

For production applications, consider deferring file cleanup until after transaction commit and designing a retryable cleanup process when file deletion must be reliable.

Part 14: Django Signals Best Practices

1. Keep Receivers Small

A signal receiver should generally perform a focused task.

Avoid placing a large business workflow inside a receiver.

2. Avoid Unnecessary Database Queries

Signals may execute frequently.

Avoid adding expensive queries to receivers without a clear need.

3. Avoid Recursive Saves

Calling `instance.save()` from a `post_save` receiver can trigger the same receiver again.

Use an appropriate alternative, such as a queryset update or explicit application logic.

4. Use `transaction.on_commit()` for External Side Effects

Emails, external API calls, and background task scheduling may need to happen only after a successful database commit.

Use `transaction.on_commit()` when appropriate.

5. Register Signals in `AppConfig.ready()`

Keep signal imports in the application's initialization process.

Do not import and connect the same receiver repeatedly from views or request-handling functions.

6. Use Explicit Sender Restrictions

Prefer:

```
@receiver(post_save, sender=Product)
def product_saved(sender, instance, **kwargs):
    ...
```

This limits the receiver to the intended model.

7. Do Not Rely on Signals for Every Business Rule

If a workflow must always occur when an order is placed, a service function may make that dependency clearer.

For example:

```
def place_order(user, items):
    # Validate the order.
    # Create the order.
    # Create the order items.
    # Register post-commit work.
    ...
```

This makes the business workflow explicit.

8. Test Signal Receivers

Use automated tests to verify:

- Whether the receiver runs.
- Whether it runs only for the intended sender.
- Whether it behaves correctly for created and updated instances.
- Whether transactions and rollbacks are handled correctly.
- Whether recursive behavior is avoided.

9. Avoid Irreversible Side Effects Before Commit

A database transaction cannot automatically undo an email, an external API request, or an uploaded file deletion.

Coordinate these operations carefully.

10. Be Careful with Weak References

Django signal connections normally use weak references to receivers where possible.

For a receiver that might otherwise be garbage-collected, Django supports `weak=False` when connecting the receiver directly.

Example:

```
post_save.connect(
    my_receiver,
    sender=Product,
    weak=False,
)
```

Use this only when necessary. Most module-level receiver functions remain available for the lifetime of the application.

Part 15: Common Django Signals Mistakes

Mistake 1: Forgetting to Register Signals

A receiver defined in `signals.py` will not run unless its module is imported and the receiver is registered.

Solution: Import the signal module through `AppConfig.ready()`.

Mistake 2: Creating Duplicate Profiles

A user profile may already exist when the signal runs.

Solution: Use appropriate uniqueness constraints and, where suitable, `get_or_create()`.

Mistake 3: Infinite Recursion

Calling `instance.save()` inside a receiver can trigger the same signal repeatedly.

Solution: Avoid recursive saves or carefully control the update flow.

Mistake 4: Assuming `post_save` Means Transaction Committed

The outer transaction may still be active.

Solution: Use `transaction.on_commit()` when the action must happen only after a successful commit.

Mistake 5: Assuming Bulk Operations Trigger Save Signals

`bulk_create()`, `bulk_update()`, and `QuerySet.update()` bypass the normal per-instance save lifecycle.

Solution: Handle the required business logic explicitly when using these operations.

Mistake 6: Putting Too Much Logic Inside Signals

Large receivers make business workflows difficult to follow.

Solution: Move complex operations into service functions and keep receivers small.

Mistake 7: Deleting Files Immediately Inside a Transaction

External file deletion cannot necessarily be rolled back with a database transaction.

Solution: Defer cleanup until commit and use a retryable cleanup mechanism if reliability is important.

Part 16: Django Signals Interview Questions

Q1. What are Django Signals?

Django Signals allow components of an application to respond to events that occur elsewhere in the application.

Q2. Why do we use Django Signals?

They are useful for event-driven tasks such as creating related objects, recording events, and scheduling notifications.

Q3. What is a signal receiver?

A receiver is a function or callable that executes when its connected signal is sent.

Q4. What is the difference between `pre_save` and `post_save`?

`pre_save` is sent before the model save operation, while `post_save` is sent at the end of that operation.

Q5. What is the purpose of the `created` argument?

It indicates whether the `post_save` operation created a new database record.

Q6. How do you automatically create a user profile?

Register a `post_save` receiver for the user model and create a profile when `created=True`.

Q7. Where should signal receivers be registered?

A common approach is to define them in `signals.py` and import that module through the application's `AppConfig.ready()` method.

Q8. What is `m2m_changed`?

It is a signal that notifies receivers about changes to ManyToMany relationships made through the relationship manager.

Q9. What is `dispatch_uid`?

It is an identifier used to distinguish signal receiver registrations and help prevent duplicate connections.

Q10. What is a custom signal?

A custom signal is an application-defined event that can notify registered receivers.

Q11. What is the difference between `send()` and `send_robust()`?

`send()` propagates receiver exceptions. `send_robust()` catches receiver exceptions and returns them as part of the response list.

Q12. Do `QuerySet.update()` and `bulk_create()` trigger `post_save`?

No. These operations bypass the normal per-instance save lifecycle.

Q13. Can Django Signals cause infinite recursion?

Yes. For example, calling `instance.save()` inside a `post_save` receiver may trigger that receiver repeatedly.

Q14. Why is `transaction.on_commit()` important?

It allows callbacks to run only after the relevant database transaction successfully commits.

Q15. Are Django Signals always the best way to implement business logic?

No. Signals are useful for decoupled event handling, but explicit service functions are often easier to understand and maintain for critical business workflows.

Q16. Does `post_delete` guarantee that a deletion transaction has committed?

No. The deletion operation may occur inside a larger transaction that later rolls back.

Q17. What happens if a receiver raises an exception?

With ordinary `send()`, the exception propagates and may interrupt the operation that sent the signal. The final effect on database changes depends on the transaction context.

Q18. Can a Django model have multiple signal receivers?

Yes. Multiple receivers can listen to the same signal, subject to their connection configuration.

Part 17: Quick Revision Table

Requirement	Recommended Signal or Approach

Prepare data before saving	`pre_save`
Respond to a model save	`post_save`
Prepare for model deletion	`pre_delete`
Respond to model deletion	`post_delete`
Respond to ManyToMany changes	`m2m_changed`
Respond to model initialization	`pre_init` / `post_init`
Create a profile after user creation	`post_save`
Prevent duplicate receiver	`dispatch_uid`

registration	
Define application-specific events	Custom signals
Run work after transaction commit	`transaction.on_commit()`
Avoid recursive model saves	Explicit logic or carefully controlled updates
Avoid unnecessary signal execution	Restrict the sender
Handle reliable external notifications	Post-commit callbacks and, where needed, a task queue or transactional outbox

---

Final Revision Checklist

- Understand what Django Signals are.
- Understand signals, senders, receivers, and instances.
- Use `pre_save` and `post_save`.
- Understand `pre_delete` and `post_delete`.
- Understand `m2m_changed` and its actions.
- Create and register signal receivers correctly.
- Automatically create related objects when appropriate.
- Avoid recursive signal execution.
- Understand `dispatch_uid`.
- Create and send custom signals.
- Understand the difference between `save()` and bulk operations.
- Use `transaction.on_commit()` for post-commit actions.
- Understand why external side effects need separate reliability considerations.
- Test receivers and handle exceptions appropriately.
- Know when a service function is preferable to a signal.

Final Advice: Django Signals are powerful tools for event-driven programming, but they should be used carefully. Keep receivers small, register them correctly, understand transaction boundaries, and avoid hiding essential business logic behind unexpected side effects.